Ruby · brief per Claude Code

Nov 5, 2026 · @Dennis

A cosa serve

Il brief porta Ruby dalla versione di oggi alle 25 novità del template, in cinque fasi, lavorando solo su Ruby.

- **Come si usa:** incolla in Code il messaggio dell'ultima sezione per partire con la fase 1. Per le fasi successive, incolla la riga della fase e le righe delle sue funzioni.
- **Perimetro:** solo Ruby, cioè l'attuale cartella `Eve/`, il codice dell'assistente in `Sources/Prisma/`, le istruzioni, gli schemi e gli esempi. Il resto di Prisma non si tocca, salvo esporre a Ruby le funzioni che le servono come strumenti.
- **Riferimento grafico:** il template navigabile Ruby · le 25 novità. Ogni funzione ha la sua scena; dati, documenti e polizze sono fac-simile.
- **Punto di partenza (dalla scheda di Eve nel repository):** Qwen 3.5 4B via Ollama, modalità lavoro e conversazione, lettura dei PDF e testo dalle scansioni, scenari «e se», copilota dell'incontro, comandi a regole, contesto di 8.192 token, ponte con l'iPad. Erano già in programma la ricerca nei documenti, le azioni con chiamata di strumenti, la memoria persistente e un motore esterno.

Le regole fisse

Otto regole valgono per tutte le funzioni: una funzione che ne viola una non è finita.

1. **I dati dei clienti non escono dal Mac.** Il motore esterno (fase 5) riceve solo domande della modalità conversazione, senza schede, documenti o numeri dei clienti. Questa regola non si spegne dalle impostazioni.
2. **Prima di agire, conferma.** Ogni azione che cambia qualcosa (agenda, schede, pratiche, messaggi, posizioni) passa dalla scheda di conferma. Ruby non invia, non cancella e non firma da sola.
3. **I numeri li calcola Prisma.** Il modello sceglie lo strumento e i parametri, non fa conti. Ogni cifra mostrata arriva dal risultato di uno strumento.
4. **Ogni dato ha la sua fonte.** Le affermazioni su un documento indicano documento, pagina o articolo. Se l'informazione non c'è, Ruby lo dice e non la inventa.
5. **Prima il codice, poi il modello.** Quello che si controlla con regole (date, scadenze, parole come «garantito», numeri che non tornano) lo fa il codice. Il modello spiega e scrive.

6. **Uscite strutturate.** Il modello risponde in JSON con schemi definiti, come già avviene per scenari e copilota, e l'output non valido si scarta. Con il modello Apple si usa la guided generation.
7. **Il Mac ha 8 GB.** Un solo modello grande caricato alla volta. Qwen 4B si scarica dopo 10 minuti di inattività, come oggi. Il modello di ricerca si carica solo per indicizzare e cercare.
8. **Funziona senza internet.** Fanno eccezione solo il motore esterno e l'addestramento della fase 5. Si resta compatibili con macOS 13: le funzioni del modello Apple richiedono macOS 26 e restano dietro i controlli di disponibilità, con ripiego su Qwen.

I motori di Ruby

Ruby usa cinque motori gratuiti sul Mac, più un motore esterno facoltativo. Ogni funzione dichiara quale usa.

Motore	Dove gira	Memoria	Compito	Funzioni
Qwen 3.5 4B, via Ollama	Mac	circa 3,5 GB mentre risponde	capire le richieste, chiamare gli strumenti, scrivere, ragionare a passi	1, 2, 3, 6, 8, 10, 12, 14, 15, 16, 17, 20, 24
Modello Apple (Foundation Models)	Mac, da macOS 26	gestita dal sistema	comandi rapidi, riconoscere un documento, etichette, riassunti brevi	9, 23 e lo smistamento
Qwen3-Embedding 0.6B, via Ollama	Mac	meno di 1 GB, solo quando serve	ricerca per significato in documenti, norme, note e ricordi	6, 7, 8, 12, 13, 18, 19
Vision e Speech di Apple	Mac	gestita dal sistema	testo da scansioni e foto, trascrizione delle note vocali	9, 16
Unsloth	GPU gratuita in cloud, ogni tanto	nessuna sul Mac	addestramento LoRA del 4B su esempi anonimi e approvati	22

Motore	Dove gira	Memoria	Compito	Funzioni
Solo codice	Prisma	nessuna	regole, calcoli, ricerca degli orari liberi, confronto riga per riga	4, 5, 7, 11, 19, 21
Motore esterno (facoltativo)	server proprio o servizio a pagamento	nessuna sul Mac	solo conversazioni generali	25

Lo smistamento: una funzione di codice sceglie il motore. Prima le regole a parole (i comandi di oggi), poi il modello Apple, infine Qwen. Se la risposta veloce non è valida, la domanda passa a Qwen.

Prima di usarli, Code deve controllare in Ollama i nomi esatti dei modelli (per esempio `qwen3.5:4b` e `qwen3-embedding:0.6b`) e misurare la memoria usata sul Mac.

Le cinque fasi

Si costruisce in cinque fasi, e ognuna si chiude con la build verde su GitHub Actions e le prove della fase superate. L'ordine segue le dipendenze: prima strumenti e conferma, poi memoria e ricerca, che servono a quasi tutto il resto.

1. **Fase 1 · Le fondamenta.** Eve diventa Ruby ovunque: interfaccia, istruzioni («sei Ruby»), cartella `Eve/` → `Ruby/`, chiavi delle impostazioni, con una migrazione che conserva conversazioni e preferenze. Nasce il registro degli strumenti: ogni azione di Prisma che Ruby può usare (agenda, clienti, pratiche, messaggi, posizioni, studio, simulatore) con nome, descrizione e schema JSON. Nasce la scheda di conferma unica.
   - Funzioni: 1 Fallo tu, 2 Chiedi alla tua clientela, 19 Mai a caso.
2. **Fase 2 · Memoria e ricerca.** Arrivano Qwen3-Embedding 0.6B e l'indice locale dei documenti dei clienti e della biblioteca delle norme, la memoria persistente con la sua pagina, le lezioni dalle correzioni e il contesto più ampio con il riassunto progressivo.
   - Funzioni: 6 Il cliente in 30 secondi, 9 Lascia cadere qui, 13 Il codice in tasca, 18 La memoria, 21 Impara dalle correzioni, 24 Il filo lungo.
3. **Fase 3 · Lettura e controllo.** Ruby legge, confronta e controlla i documenti, sempre con la fonte.
   - Funzioni: 8 Le promesse, 10 La seconda lettura, 11 Trova le differenze, 12 Cosa non copre, 16 Il resoconto dell'incontro.
4. **Fase 4 · Ruby che lavora per te.** Le funzioni che combinano strumenti, memoria e ricerca.

    - Funzioni: 3 Le routine, 4 L'agenda che si organizza, 5 Il tagliando dei dati, 7 Il sussurro, 14 Il mercato per i tuoi clienti, 15 Scrive come te, 17 Tre livelli, 20 Ragiona a passi.
5. **Fase 5 · Più intelligente.** Lo smistamento tra modello Apple e Qwen, l'addestramento su misura e la predisposizione del motore esterno, che resta non collegato.
    - Funzioni: 22 Ruby su misura, 23 Due velocità, 25 Il cervello grande.

Le 25 funzioni

Per ogni funzione la tabella dice cosa fa, quale motore usa, cosa fa il codice e quando è pronta. I numeri sono quelli del template.

#	Funzione e cosa fa	Fase	Motore	Il codice fa	Pronta quando
1	**Fallo tu**: una richiesta a parole diventa azioni in qualsiasi app	1	Qwen 4B con strumenti	registro degli strumenti, scheda di conferma, esecuzione	«sposta l'incontro e avvisala» produce due azioni, eseguite solo dopo «Conferma»
2	**Chiedi alla tua clientela**: domande su tutti i clienti insieme	1	Qwen 4B, che traduce in filtri JSON	interrogazione dei dati, tabella, «Crea attività»	mostra i filtri capiti; il risultato è uguale alla stessa ricerca fatta a mano
3	**Le routine**: sequenze salvate, lanciate con una frase	4	Qwen 4B	esecuzione passo per passo, routine create a parole	«prepara l'incontro con Marco» esegue i passi e lascia la scaletta
4	**L'agenda che si organizza**: orari giusti, spostamenti in blocco, due orari proposti al cliente	4	Qwen 4B per capire, codice per cercare	ricerca degli orari liberi vicino agli incontri in studio	propone due orari senza sovrapposizioni e rispetta le preferenze (mai prima delle 9:30)
5	**Il tagliando dei dati**: controllo notturno dell'archivio	4	solo codice, Qwen per i testi	regole su doppioni, campi vuoti, consensi e documenti scaduti	ogni correzione chiede conferma; nulla si cancella da solo
6	**Il cliente in 30 secondi**: la relazione raccontata in cinque punti	2	Qwen 4B con ricerca	raccoglie scheda, incontri, messaggi e ricordi	cinque sezioni fisse, ognuna con la sua fonte
7	**Il sussurro**: due o tre cose da sapere quando apri una scheda	4	codice con ricerca	regole di priorità su scadenze, promesse e consensi	al massimo tre voci, ognuna con un'azione

#	Funzione e cosa fa	Fase	Motore	Il codice fa	Pronta quando
8	**Le promesse**: gli impegni presi, trovati in note, resoconti e messaggi	3	Qwen 4B (estrazione in JSON) con ricerca	elenco, scadenze, promemoria	ogni promessa riporta la frase da cui viene
9	**Lascia cadere qui**: documento trascinato, riconosciuto e archiviato	2	Vision e modello Apple, Qwen se manca	archiviazione nel cliente giusto, date, promemoria	i dati letti si confermano prima di salvarli
10	**La seconda lettura**: rilettura prima dell'invio e bozza della motivazione di adeguatezza	3	regole, poi Qwen 4B	parole a rischio, numeri confrontati con Prisma, allegati	segnala «garantito», cifre diverse dalla scheda e KID mancante; la valutazione finale resta all'utente
11	**Trova le differenze**: due documenti a confronto	3	codice per confrontare, Qwen per la sintesi	allineamento delle voci, tabella prima/ora	tabella delle voci cambiate e una sintesi dell'effetto sul cliente
12	**Cosa non copre**: esclusioni, carenze, franchigie e massimali di una polizza	3	Qwen 4B con ricerca	rimando all'articolo, vista con le parole del cliente	ogni voce cita l'articolo; senza articolo la voce non compare
13	**Il codice in tasca**: risposte sulle norme	2	ricerca e Qwen 4B	biblioteca delle norme caricata dall'utente	cita sempre l'articolo; se la norma non risponde, lo dice
14	**Il mercato per i tuoi clienti**: un evento di mercato e i clienti che tocca	4	codice per le regole, Qwen per i testi	dati BCE ed Eurostat già in Prisma, regole sui portafogli	elenco con motivo e azione per cliente, bozze nello stile dell'utente
15	**Scrive come te**: lo stile imparato dai messaggi inviati	4	Qwen 4B con regole ed esempi di stile	profilo di stile visibile e modificabile	ogni bozza segue le regole del profilo; niente parte senza conferma
16	**Il resoconto dell'incontro**: verbale dalla nota vocale e dal copilota	3	Speech e Qwen 4B	crea attività e promesse	da rivedere e firmare; consenso alla registrazione registrato
17	**Tre livelli**: la stessa spiegazione per te, per il cliente e in una pagina	4	Qwen 4B	numeri da Prisma, pagina stampabile	stessi numeri nelle tre versioni

#	Funzione e cosa fa	Fase	Motore	Il codice fa	Pronta quando
18	**La memoria**: ricordi su di te e sui clienti	2	ricerca e archivio locale cifrato	pagina dei ricordi, interruttore	ogni ricordo si vede, si corregge e si cancella; mai numeri di documenti, conti o password
19	**Mai a caso**: la fonte di ogni dato	1	codice	confronto delle cifre con Prisma, citazioni	ogni cifra ha la sua fonte; «non trovato» quando manca
20	**Ragiona a passi**: un piano visibile, eseguito un passo alla volta	4	Qwen 4B con strumenti	simulatore di Prisma, interruttore «mostra il ragionamento»	al massimo cinque passi, ognuno con il suo strumento
21	**Impara dalle correzioni**: pollici e correzioni diventano lezioni	2	codice (archivio delle lezioni)	lezioni aggiunte alle istruzioni prima di rispondere	una lezione salvata vale dalla risposta successiva
22	**Ruby su misura**: addestramento sui tuoi esempi	5	Unsloth su GPU gratuita	anonimizzazione, esame di prova, ritorno alla versione precedente	il nuovo modello sostituisce il vecchio solo se supera l'esame
23	**Due velocità**: smistamento tra modello Apple e Qwen	5	modello Apple e Qwen 4B	smistamento, ripiego su Qwen	sotto macOS 26 tutto va a Qwen, senza errori
24	**Il filo lungo**: più contesto e un riassunto progressivo	2	Qwen 4B	riassunto progressivo, contesto da 8.192 a circa 32.000 token se la memoria del Mac regge	risponde su fatti detti all'inizio della conversazione; uso della memoria misurato
25	**Il cervello grande**: aggancio a un motore esterno	5	esterno, non collegato	regole bloccate, etichetta del motore sotto ogni risposta	i dati dei clienti non vengono mai inviati

Da incollare in Code

Copia il messaggio nella chat di Code dedicata a Ruby, poi incolla sotto tutto il brief (oppure esportalo in Markdown e allegalo).

Lavoriamo solo su Ruby, l'assistente di Prisma che prima si chiamava Eve. Il resto dell'app non si tocca, salvo esporre a Ruby come strumenti le funzioni che le servono.

Sotto trovi il brief completo: salvalo nel repository come Ruby/BRIEF.md e seguilo. Il template navigabile delle 25 novità te lo allego: salvalo come Design/template-ruby.html e usalo come riferimento grafico per ogni funzione.

Regole fisse (valgono sempre): i dati dei clienti non escono dal Mac; ogni azione passa da una scheda di conferma; i numeri li calcola Prisma, mai il modello; ogni dato ha la sua fonte e, se manca, Ruby lo dice; prima il codice, poi il modello; risposte del modello in JSON con schemi; un solo modello grande caricato alla volta (il Mac ha 8 GB); compatibilità con macOS 13 e funzioni Apple dietro i controlli di disponibilità.

Fase 1 · Le fondamenta:
1. Rinomina Eve in Ruby ovunque (interfaccia, istruzioni, cartella Eve/ → Ruby/, chiavi delle impostazioni) con una migrazione che conserva conversazioni e preferenze.
2. Crea il registro degli strumenti: ogni azione di Prisma che Ruby può usare (agenda, clienti, pratiche, messaggi, posizioni, studio, simulatore) con nome, descrizione e schema JSON degli argomenti, usando la chiamata di strumenti di Qwen 3.5 via Ollama.
3. Crea la scheda di conferma unica: elenco delle azioni con app, titolo e dettaglio, eseguite solo dopo «Conferma».
4. Costruisci le funzioni 1 Fallo tu, 2 Chiedi alla tua clientela e 19 Mai a caso, come descritte nella tabella del brief.
5. Aggiungi i casi di prova con il server finto già usato nei test e tieni verde la build su GitHub Actions.

Prima di scrivere codice, riassumimi il piano della fase 1 in dieci righe e dimmi cosa ti manca.